

JAVA FONDAMENTAUX

Présenté par :
Xavier TABUTEAU

JDBC

JDBC

Cela signifie Java Database Connectivity. C'est une API (Application Programming Interface) Java pour connecter et exécuter des requêtes sur une base de données relationnelle (BDDR). Il fait partie de JavaSE (Java Standard Edition). Cette API utilise des pilotes JDBC pour se connecter à une BDDR.

L'API JDBC permet d'accéder aux données stockées dans n'importe quelle BDDR. Il est possible d'insérer, mettre à jour, supprimer et lire des données d'une BDDR. C'est comme Open Database Connectivity (ODBC) de Microsoft.

Avant JDBC, l'API ODBC était utilisée, mais celle-ci utilise un pilote écrit en C (c'est à dire dépendant de la plateforme et non sécurisé). C'est pourquoi Java a défini sa propre API (JDBC) qui utilise des pilotes JDBC (écrits en langage Java et par conséquent sécurisé et non dépendant de la plateforme).

JDBC

JDBC

Il existe 5 étapes pour connecter une application Java à une BDDR à l'aide de JDBC. Ces étapes sont les suivantes :

- Enregistrer la classe de pilote
- Créer une connexion
- Créer un objet Statement / PreparedStatement
- Exécuter des requêtes
- Fermer la connexion

Pour pouvoir utiliser JDBC il faut :

Un Système de Gestion de Base de Données Relationnelle (SGBDR), télécharger le pilote correspondant au SGBDR utilisé, puis l'intégrer dans le projet via notre IDE.

JDBC

JDBC

• Transaction

Avec JDBC, par défaut, une connexion à une BDD est auto commit. C'est-à-dire que toutes les requêtes SQL sont automatiquement validées. Il est possible de désactiver ce fonctionnement pour effectuer une ou plusieurs requêtes et valider ou annuler l'ensemble de ces requêtes. C'est ce qu'on appelle une transaction. La bonne pratique est de réactiver l'auto validation des requêtes lorsqu'on ne fait plus de transaction.

Propriété d'une transaction (ACID)

Atomicité : Tout ou rien.
Cohérence : L'état de la BDD reste valide.
Isolation : Transactions indépendantes.
Durabilité : Une fois validée, elle persiste.

JDBC

JDBC

- **Pool de connexion**

Une connexion JDBC prend du temps et consomme pas mal de ressources.

Un pool de connexion permet de pré-cr  er plusieurs connexions, de les r  utiliser, g  rer automatiquement leurs ouvertures / fermetures, le timeout ainsi que le nombre maximum de celles-ci.

JDBC

JDBC

Pour le SGBDR nous utiliserons Laragon (version complète), c'est un logiciel gratuit, local, intégrant les serveurs : MySQL, PHP, Apache et NodeJS.

Lien de téléchargement : <https://laragon.org/download/index.html>

Lien de téléchargement du pilote JDBC pour MySQL :
<https://dev.mysql.com/downloads/connector/>

Lien HikariCP (Pool de connexion) :
<https://repo1.maven.org/maven2/com/zaxxer/HikariCP/6.2.1/HikariCP-6.2.1.jar>

Lien SLF4J API et SLF4J Simple (Simple Logging Facade for Java) obligatoire pour HikariCP :
<https://repo1.maven.org/maven2/org/slf4j/slf4j-api/2.0.9/slf4j-api-2.0.9.jar>
<https://repo1.maven.org/maven2/org/slf4j/slf4j-simple/2.0.9/slf4j-simple-2.0.9.jar>

Dézipper et stocker le pilote JDBC pour MySQL (le fichier avec l'extension « .JAR ») dans l'emplacement voulu sur votre PC.

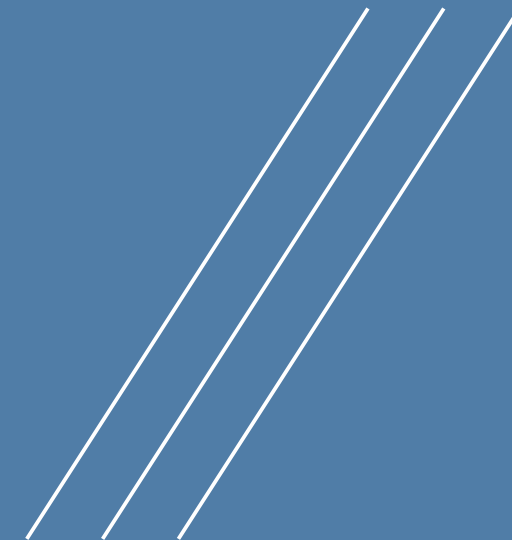
Procédure pour intégrer le pilote JDBC MySQL dans notre projet via l'IDE NetBeans :

Cliquer sur l'onglet Projects, dans la fenêtre latérale, faire un clic droit sur librairies, puis choisir « add JAR Folder... ».

Une fenêtre s'ouvre, sélectionner le fichier « .JAR » télécharger et stocké précédemment et cliquer sur le bouton « Ouvrir ».

JDBC.java

JAVA FONDAMENTAUX



Présenté par
Xavier TABUTEAU